

Znajdowanie miejsc zerowych metodą bisekcji oraz całkowanie numeryczne metodami: Monte Carlo i trapezów

Jakub Sikorski 354584

2025-06-04

Spis treści

1	Cel raportu:	2
2	Metoda Bisekcji:	2
2.1	Opis metody:	2
2.2	Implementacja i wyniki:	3
2.3	Wnioski:	3
2.4	Podsumowanie:	4
3	Metoda Monte Carlo	5
3.1	Opis metody:	5
3.2	Implementacja i wyniki:	6
3.3	Wnioski:	7
3.4	Podsumowanie:	7
4	Metoda Trapezów	8
4.1	Opis metody:	8
4.2	Implementacja i wyniki:	8
4.3	Wnioski:	9
4.4	Podsumowanie:	10
5	Bibliografia:	11

1 Cel raportu:

Celem niniejszego raportu jest przedstawienie, porównanie oraz analiza trzech wybranych metod numerycznych: całkowania metodą trapezów, całkowania metodą Monte Carlo oraz wyznaczania miejsc zerowych funkcji za pomocą metody bisekcji. Każda z metod została zastosowana do odpowiednio dobranych funkcji, a następnie oceniono jej skuteczność, dokładność oraz wpływ kluczowych parametrów (takich jak liczba podziałów czy liczba losowań) na otrzymane wyniki. Raport ma na celu nie tylko zapoznanie czytelnika z zasadą działania każdej z metod, ale także ukazanie ich zalet, ograniczeń oraz przydatności w różnych zastosowaniach obliczeniowych.

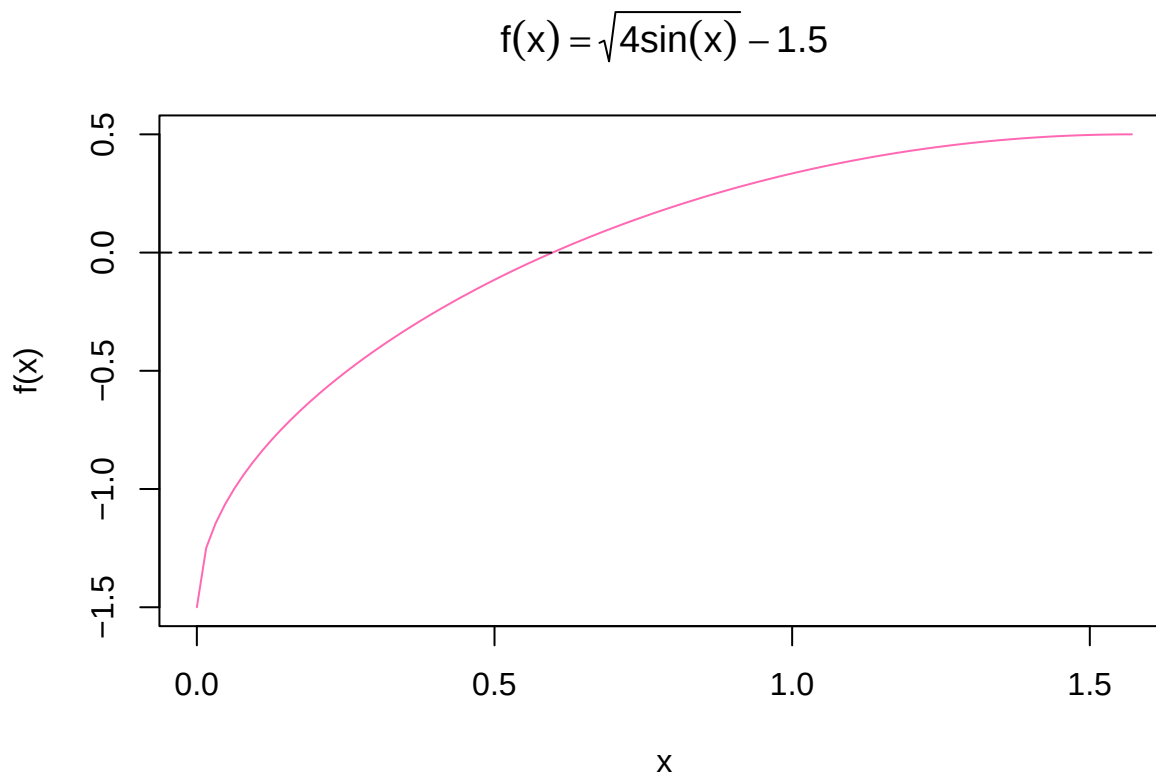
2 Metoda Bisekcji:

2.1 Opis metody:

Metoda bisekcji (dzielenia przedziału na pół) służy do znajdowania miejsc zerowych funkcji. Zakłada, że funkcja zmienia znak na końcach przedziału i iteracyjnie dzieli go na pół, wybierając ten podprzedział, w którym zachodzi zmiana znaku, aż do osiągnięcia zadanej dokładności. Wzór którego będziemy używać do zawężania przedziału na którym pracujemy to:

$$|a - b| > \varepsilon$$

Wykres opisujący tą metodę na przedziale $[0, \frac{\pi}{2}]$ wygląda następująco:



2.2 Implementacja i wyniki:

Kod wykonujący potrzebne nam obliczenia oraz zwracający wynik wygląda następująco:

```
a<-0
b<-pi/2
e<-0.01
sinus<-function(x){
  sqrt(4*sin(x))-1.5
}

bisekcja<-function(a,b,e){
  c<-c()
  n<-0
  while(abs(a-b)>e){

    if(sinus((a+b)/2)*sinus(a)< 0){
      b<-(a+b)/2
    }else{a<-(a+b)/2
    }
    n<-n+1
  }
  c[1]<-(a+b)/2
  c[2]<-n
  c
}

wynik1<-bisekcja(a,b,e)
wynik1[1]
```

```
## [1] 0.5982525
```

Liczba iteracji potrzebnych do osiągnięcia punktu z dokładnością do ε wynosi: 8.

2.3 Wnioski:

Posiadając już kod pozwalający nam obliczać przybliżone miejsca zerowe, możemy obliczyć je w zależności od naszego ε oraz sprawdzić ile iteracji jest nam do tego potrzebnych. Poniższa tabela porównuje wyniki dla $\varepsilon = 10^{-k}$, $k \in 3, 4, 5, 6$:

Table 1: Podsumowanie

Wartosci_k	Wyniki	Iteracje
3	0.5971020	11
4	0.5974376	14
5	0.5974046	18
6	0.5974065	21

2.4 Podsumowanie:

Metoda bisekcji jest bardzo stabilna i gwarantuje zbieżność, jeśli tylko funkcja spełnia warunek zmiany znaku na końcach przedziału. Jej główną zaletą jest niezawodność, natomiast wadą – stosunkowo wolna zbieżność w porównaniu do metod wykorzystujących pochodne (np. Newtona-Raphsona).

3 Metoda Monte Carlo

3.1 Opis metody:

Metoda Monte Carlo opiera się na losowym próbkowaniu wartości funkcji w obrębie przedziału całkowania i uśrednianiu tych wyników. Przybliżona wartość całki jest proporcjonalna do średniej wartości funkcji oraz długości przedziału. Metoda ta wykorzystuje narzędzia rachunku prawdopodobieństwa i statystyki. Wzór ogólny prezentuje się następująco:

$$\int_a^b f(x)dx \approx (b-a) \frac{1}{N} \sum_{i=1}^N f(x_i)$$

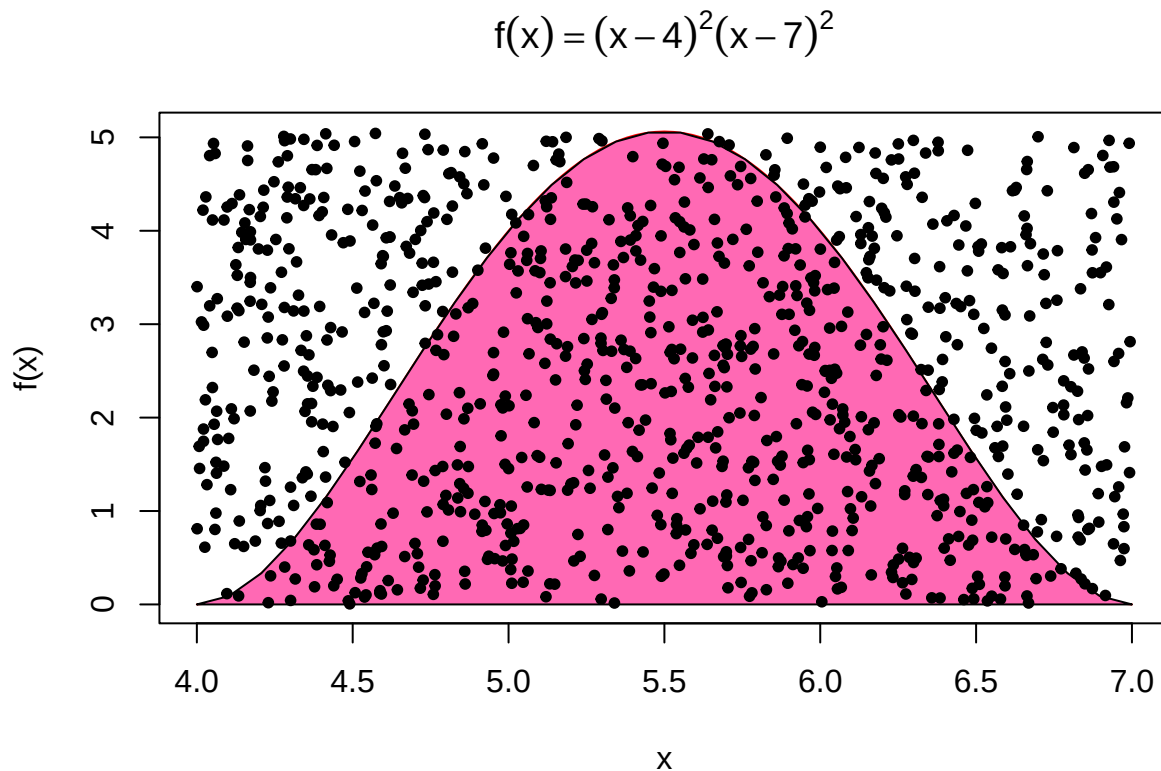
N - liczba losowo wygenerowanych punktów

$x_i \in [a, b]$ - losowe punkty

$f(x_i)$ - wartość funkcji w wylosowanym punkcie

$(b-a)$ - długość całkowanego przedziału

Funkcja którą będziemy się zajmować określona jest wzorem $(x-4)^2(x-7)^2$ na przedziale $[4,7]$ dla $N = 1000$. Ukazane jest to na poniższym wykresie:



3.2 Implementacja i wyniki:

Kod korzystający z tej metody 10-cio krotnie (w celu zoptymalizowania wyników) dla $N = 1000$ wygląda następująco:

```
a<-4
b<-7
N1<-1000
fx<-function(x){
  (x-4)^2 * (x-7)^2
}
x_los<-runif(N1,min = a,max = b)
max_fx <- max(fx(x_los))
y_los<-runif(N1,min = 0,max = max_fx)
srednia1<-c()
for(j in 1:10){
  wynikmc1<-0
  x_los<-runif(N1,min = a,max = b)
  y_los<-runif(N1,min = 0,max = max_fx)
  for(i in 1:N1){
    if(y_los[i]<=fx(x_los[i])){
      wynikmc1<-wynikmc1+1
    }
  }
  srednia1[j]<-wynikmc1
}
wynik1k<-(sum(srednia1)/10/N1)*(b-a)*max_fx
```

Jeśli chcemy, aby nasze wyniki były jeszcze dokładniejsze możemy zwiększyć nasze N , kod dla $N = 10000$ prezentuje się tak:

```
a<-4
b<-7
N2<-10000
fx<-function(x){
  (x-4)^2 * (x-7)^2
}
x_los<-runif(N2,min = a,max = b)
max_fx <- max(fx(x_los))
y_los<-runif(N2,min = 0,max = max_fx)
srednia2<-c()
for(j in 1:10){
  wynikmc2<-0
  x_los<-runif(N2,min = a,max = b)
  y_los<-runif(N2,min = 0,max = max_fx)
  for(i in 1:N2){
    if(y_los[i]<=fx(x_los[i])){
      wynikmc2<-wynikmc2+1
    }
  }
  srednia2[j]<-wynikmc2
}
wynik10k<-(sum(srednia2)/10/N2)*(b-a)*max_fx
```

3.3 Wnioski:

Gdy mamy już kod generujący nam wyniki korzystając z metody Monte Carlo, możemy przyrównać je do wyniku otrzymanego z liczenia metodą analityczną, co przedstawia poniższa tabela:

Table 2: Podsumowanie metody Monte Carlo

Wyniki_N1000	Wyniki_N10000
8.125312	8.145056
8.064562	8.096456
8.201250	8.035706
8.520188	8.158725
8.429063	8.055450
7.684875	8.101012
8.064562	8.117719
7.988625	8.108606
7.821562	8.181506
8.201250	8.189100

Table 3: Porównanie wyników

Srednia_N1000	Srednia_N10000	Wynik_analityczny
8.110123	8.118934	8.1

3.4 Podsumowanie:

Monte Carlo jest bardzo uniwersalną metodą, szczególnie przydatną przy całkowaniu funkcji w wielu wymiarach. Choć jej dokładność rośnie wolno (proporcjonalnie do pierwiastka z liczby losowań), jest łatwa do zaimplementowania i skuteczna tam, gdzie inne metody zawodzą. Działa dobrze nawet dla funkcji nieciągłych lub trudnych do analitycznego opisanie.

4 Metoda Trapezów

4.1 Opis metody:

Metoda trapezów to klasyczna technika numerycznego całkowania polegająca na przybliżeniu pola pod wykresem funkcji za pomocą sumy pól trapezów. Przedział całkowania dzielony jest na mniejsze podprzedziały (odcinki), a pole każdego z nich szacowane jest jako średnia arytmetyczna wartości funkcji na końcach przedziału pomnożona przez jego długość. Wzór ogólny tej metody wygląda następująco:

$$\int_a^b f(x)dx \approx \frac{h}{2}(f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n))$$

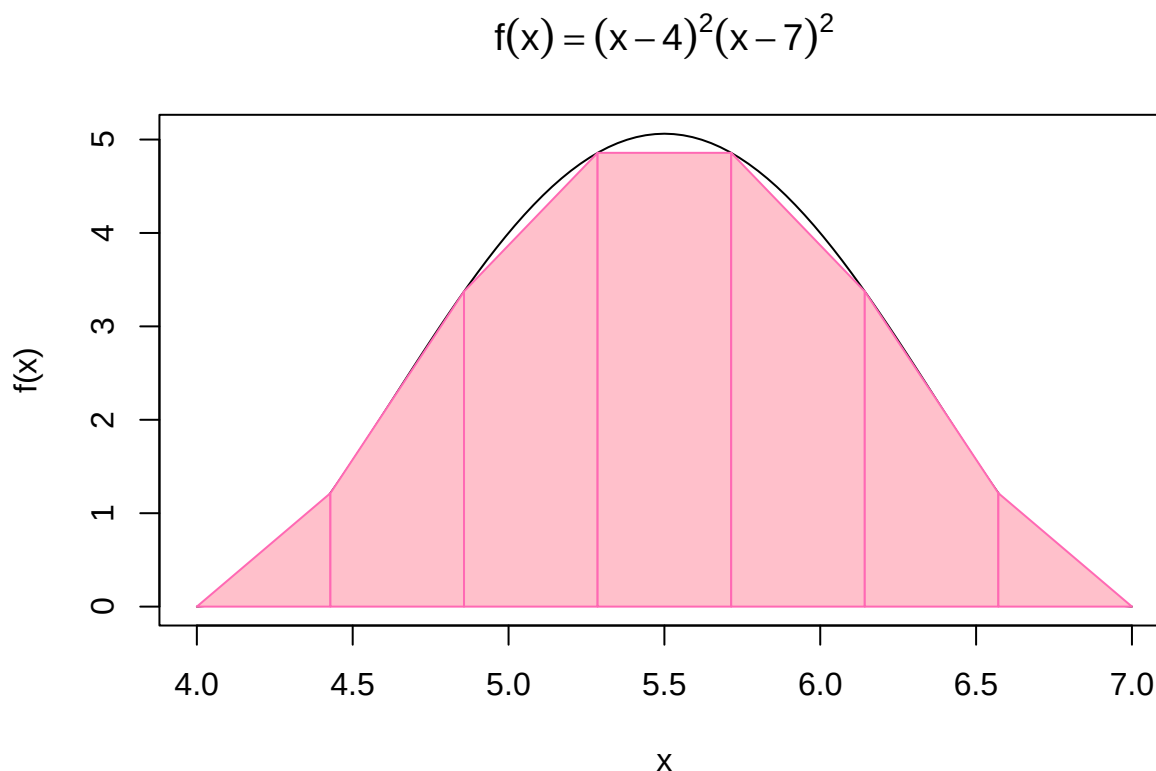
$h = \frac{b-a}{n}$ - szerokość każdego podprzedziału

x_0 - a

x_n - b

$x_i = a + i * h$ dla $i = 0, 1, 2, \dots, n$

Poniższy wykres przedstawia podział pola pod wykresem funkcji na 7 trapezów:



4.2 Implementacja i wyniki:

Kod liczący podaną całkę przy użyciu metody trapezów przy podziale na n równych odcinków wygląda następująco:

```

a<-4
b<-7
fx<-function(x){
  (x-4)^2 * (x-7)^2
}
n<-7
x<-seq(a,b,length.out=n+1)
y<-fx(x)

trapez<-function(a,b,n,fx){
  h<-(b-a)/n
  wyniktr<-(h/2)*(y[1]+2*sum(y[2:n]+y[n+1]))
  wyniktr
}
trapez(a,b,n,fx)

```

Teraz możemy przekształcić ten kod tak, żebyśmy mogli dowiedzieć się na ile trapezów musimy podzielić pole pod wykresem, aby dokładność wyniku była $< \varepsilon$ dla $\varepsilon = 0.01$. Kod będzie wyglądał następująco:

```

a<-4
b<-7
fx<-function(x){
  (x-4)^2 * (x-7)^2
}
n<-1
e<-0.01
trapez<-function(a,b,n,fx){
  x<-seq(a,b,length.out=n+1)
  y<-fx(x)
  h<-(b-a)/n
  wyniktr<-(h/2)*(y[1]+2*sum(y[2:n]+y[n+1]))
  wyniktr
}
wynik<-trapez(a,b,n,fx)

while(abs(8.1-wynik)>e){
  n<-n+1
  wynik<-trapez(a,b,n,fx)
}

```

I dowiadujemy się z niego, że wynik z niepewnością rzędu ε osiągamy dla $n = 6$.

4.3 Wnioski:

Możemy zauważyć, że nasza metoda jest bardzo dokładna już przy dość niskiej liczbie podziałów, gdyż dla $n = 6$ wynik różni się o < 0.01 od wyniku otrzymanego metodą analityczną.

Table 4: Podsumowanie metody trapezów

Wynik_dla_n3	Wynik_dla_n6	Wynik_dla_n9	Wynik_analityczny
8	8.09375	8.098765	8.1

4.4 Podsumowanie:

Metoda trapezów charakteryzuje się dobrą dokładnością dla funkcji gładkich i dobrze przybliżalnych liniowo. Jest prosta do implementacji i szybka w działaniu, jednak dla funkcji silnie zakrzywionych lub oscylujących może wymagać dużej liczby podziałów, aby osiągnąć akceptowalną dokładność.

5 Bibliografia:

Metoda Bisekcji

Całkowanie numeryczne

Metoda Monte Carlo

Skomplikowane wzory w LaTeX

W celu pisania podsumowań oraz opisów metod wykorzystałem sztuczną inteligencję:

ChatGPT

Qwen